`Package` is used when you don't specify that a method or a data variable is either private or public. If your program has only one class definition this change is transparent. If you don't specify either public or private for any feature meaning class, method or variable it can be accessed by all methods in the same package. So, if you have the following field in your class:

```
public class MyClass
{
int mySalary;
…
}
…
```

and your class is stored in java.util.*; then any other method in any class that is also stored in this package can change this variable to anything it wants. What's more, anybody can add their own class to any package. And if you have a method to exploit that variable with package access, you could do anything you wanted! So, if your program uses many classes that are stored in the same package, they can directly access each other's package-access methods and data variables. They only need to use the reference.variable to do so.

```
int minute; // minute declared without public
// or private.
Time2.minute // Other members of its class can
// directly access minute.
```

4.3 Creating a package

A package is a way to organise classes. Normally, you create a public class. If you don't define your class as public, then it's only accessible to other classes in the same package. Use the keyword package followed by the location.

```
package com.sun.java;
```

The package statement must be the first statement in your class file. Compile it in DOS using the –d option

```
javac -d Time2.java
```

4.4 Final instance variables

The principle of encapsulation is built around the idea of limiting access to variables. This "least privilege" concept can be expanded to include variables that should never be changed, or "Constants". Since this value is a constant, the compiler could optimise by replacing references to that variable with